

DSA0405-FDS

CO2AT2: ANALYTICAL PROBLEM SOLVING

-SYED MUHAFAIZ A

192472282

1. Data Preprocessing, Aggregation and Visualization

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

data = {
    "Name": [" Alice ", "Bob", "Charlie", "Diana", "Evan"],
    "Department": ["HR", "Engineering", "HR", "Engineering", "Marketing"],
    "Salary": [50000, 85000, np.nan, 92000, 60000],
    "Join_Date": ["2022-01-15", "2021-06-20", "2023-03-11",
                  "2020-11-01", "2024-02-28"]
}

df = pd.DataFrame(data)

# Preprocessing
df["Name"] = df["Name"].str.strip()
df["Salary"] = df["Salary"].fillna(df["Salary"].median())
df["Join_Date"] = pd.to_datetime(df["Join_Date"])
df["Years_of_Service"] = 2026 - df["Join_Date"].dt.year

# Aggregation
summary = df.groupby("Department").agg(
    Avg_Salary=("Salary", "mean"),
    Total_Employees=("Name", "count")
).reset_index()

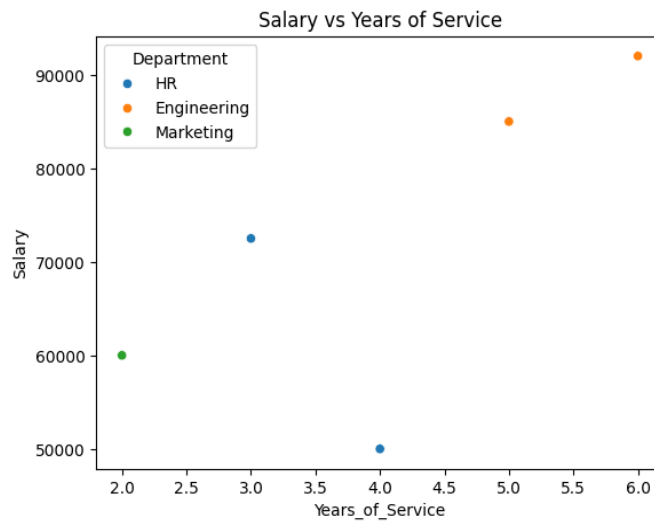
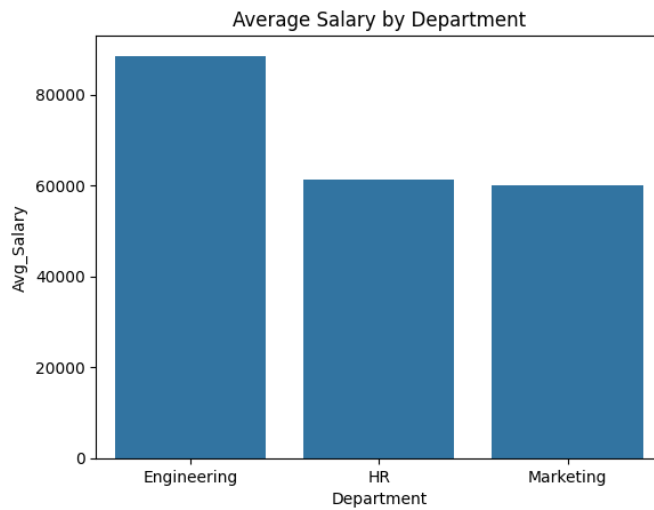
print(summary)

# Visualization
sns.barplot(data=summary, x="Department", y="Avg_Salary")
plt.title("Average Salary by Department")
plt.show()

sns.scatterplot(data=df, x="Years_of_Service", y="Salary",
                hue="Department")
plt.title("Salary vs Years of Service")
plt.show()
```

OUTPUT:

	Department	Avg_Salary	Total_Employees
0	Engineering	88500.0	2
1	HR	61250.0	2
2	Marketing	60000.0	1



2. Create a Pandas DataFrame and Add a New Row

```
import pandas as pd
```

```
df = pd.DataFrame({  
    "Name": ["John", "Emma", "Liam", "Olivia"],
```

```

    "Age": [20, 19, 21, 18],
    "Student": [True, True, False, True]
})

print(df)

new_row = pd.DataFrame(
    [
        ["Sophia", 22, False],
    ],
    columns=["Name", "Age", "Student"]
)

df = pd.concat([df, new_row], ignore_index=True)

print(df)

```

OUTPUT:

```

...      Name  Age  Student
0      John   20      True
1      Emma   19      True
2      Liam   21     False
3  Olivia   18      True
      Name  Age  Student
0      John   20      True
1      Emma   19      True
2      Liam   21     False
3  Olivia   18      True
4  Sophia   22     False

```

3. Calculate Mean, Median and Mode Using SciPy

```

import numpy as np
from scipy import stats

data = [10, 20, 30, 40, 50]

print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Mode:", stats.mode(data))

```

OUTPUT:

```

...  Mean: 30.0
     Median: 30.0
     Mode: ModeResult(mode=np.int64(10), count=np.int64(1))

```

4. Calculate Probability Using a Normal Distribution

```

from scipy.stats import norm

probability = norm.cdf(85, loc=70, scale=10)

```

```
print("Probability:", probability)
```

OUTPUT:

Probability: 0.9331927987311419

5. Perform Hypothesis Testing Using a One-Sample T-Test

```
from scipy import stats
```

```
data = [22, 25, 19, 24, 28, 30]
```

```
t_stat, p_value = stats.ttest_1samp(data, 25)
```

```
print("T-Statistic:", t_stat)
```

```
print("P-Value:", p_value)
```

OUTPUT:

T-Statistic: -0.2049800154226962

P-Value: 0.8456711339658732

6. Find Correlation Between Two Variables

```
from scipy.stats import pearsonr
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
correlation, p_value = pearsonr(x, y)
```

```
print("Correlation:", correlation)
```

OUTPUT:

Correlation: 1.0

7. Solve Linear Equations Using SciPy

```
from scipy import linalg
```

```
A = [[3, 2], [1, 2]]
```

```
B = [5, 5]
```

```
solution = linalg.solve(A, B)
```

```
print("Solution:", solution)
```

OUTPUT:

Solution: [0. 2.5]

8. Find the Minimum of a Function Using SciPy Optimization

```
from scipy.optimize import minimize
```

```
def objective(x):  
    return x**2 + 4
```

```
result = minimize(objective, x0=5)
```

```
print("Minimum value of x:", result.x)
```

OUTPUT:

Minimum value of x: [-2.62955131e-08]

9. Perform Classification Using Logistic Regression

```
from sklearn import datasets  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
from sklearn.linear_model import LogisticRegression  
from sklearn.metrics import accuracy_score
```

```
iris = datasets.load_iris()
```

```
X = iris.data  
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42  
)
```

```
scaler = StandardScaler()  
X_train = scaler.fit_transform(X_train)  
X_test = scaler.transform(X_test)
```

```
model = LogisticRegression()  
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)  
print("Accuracy:", accuracy)
```

OUTPUT:

Accuracy: 1.0

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
```

```
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(iris.data)
```

```
print("Cluster Labels:", cluster_labels)
```

Cluster Labels: [1 1]

2220222222222222222222222222202000020000

0022000020202002200000200002000200020

0 2]